

A Comprehensive Data Preprocessing and Exploratory Data Analysis of the CIC-IDS2017 Dataset

1st Miray Pınarbaşı
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
miray.pinarbasi@stu.fbu.edu.tr

2nd Sila Özgel
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
sila.ozgel@stu.fbu.edu.tr

3rd Furkan Kaya
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
furkan.kaya@stu.fbu.edu.tr

4th Umut Topatan
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
umut.topatan@stu.fbu.edu.tr

5th Göktuğ Karaca
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
goktug.karaca@stu.fbu.edu.tr

6th Ezgi Erdoğan
Department of Computer
Engineering
Fenerbahçe University
Istanbul, Türkiye
ezgi.erdogan1@stu.fbu.edu.tr

Abstract— This study proposes a structured preprocessing and machine learning–based intrusion detection approach using the CIC-IDS2017 dataset. Raw network traffic data were cleaned by removing missing and infinite values, encoding attack labels, and eliminating identifier features to prevent data leakage. Feature selection based on Information Gain and feature standardization were applied prior to modeling. Exploratory data analysis revealed strong class imbalance and skewed feature distributions. A Random Forest classifier was used to classify traffic into grouped attack categories, achieving high performance on major classes while exposing limitations in detecting rare attacks. The results highlight the importance of effective data preparation and ensemble learning for reliable intrusion detection.

Index Terms— *Intrusion Detection Systems, Cybersecurity, Machine Learning, Data Preprocessing, Exploratory Data Analysis, Random Forest, Network Traffic Analysis, CIC-IDS2017 Dataset.*

I. DATA PREPROCESSING

A. Data Loading and Cleaning

In this study, the CIC-IDS2017 dataset was subjected to a structured data loading and cleaning process to ensure reliability and numerical stability prior to any machine learning application. The dataset consists of multiple CSV files collected over different days, representing diverse benign and malicious network traffic patterns.

1) Data Loading

All CSV files belonging to the CIC-IDS2017 dataset were automatically detected and merged into a single unified dataframe using the Python pandas library. This merging process ensured that traffic records from different days were

analyzed under a consistent structure. To prevent character decoding issues commonly encountered in large-scale network logs, the cp1252 encoding scheme was applied during data ingestion. Additionally, column names were standardized by removing hidden whitespace characters to avoid potential indexing and processing errors.

2) Handling Missing and Infinite Values

Raw cybersecurity datasets frequently contain missing (NaN) and infinite (Inf) values due to packet loss, measurement errors, or abnormal traffic behavior. Such values can severely affect machine learning algorithms, leading to numerical instability or model failure.

To address this issue, all rows containing missing values were first identified and removed. Subsequently, infinite values were replaced with NaN and eliminated from the dataset using Python’s NumPy and pandas libraries. This cleaning step was critical to ensure that all remaining records contained valid and finite numerical values suitable for further analysis and modeling.

3) Label Encoding

The CIC-IDS2017 dataset includes categorical attack labels such as Benign, DDoS, and PortScan, which cannot be directly processed by most machine learning algorithms. Therefore, these categorical labels were transformed into numerical representations using the LabelEncoder provided by the Scikit-Learn library.

This transformation preserved class distinctions while enabling compatibility with supervised learning models. The encoded labels formed the target variable for subsequent intrusion detection experiments.

4) Final Dataset Export

After completing the data loading, cleaning, and label encoding steps, the processed dataset was exported as a new CSV file. This finalized dataset serves as a stable and reliable input for subsequent phases of the intrusion detection pipeline.

B. Importance of Data Cleaning in Cybersecurity Datasets

Data cleaning is a fundamental requirement in cybersecurity analytics due to the inherently noisy and heterogeneous nature of network traffic data. In intrusion detection systems, improperly handled missing or infinite values may result in misleading patterns, increased false alarm rates, or undetected attacks.

By systematically removing invalid records and standardizing label representations, the preprocessing pipeline improves data consistency, enhances model robustness, and ensures that learned patterns reflect genuine network behavior rather than artifacts of corrupted data. Consequently, effective data cleaning directly contributes to the accuracy and trustworthiness of machine learning-based intrusion detection systems.

II. EXPLORATORY DATA ANALYSIS (EDA)

A. Feature Engineering

In this study, the feature engineering workflow applied to the cleaned CIC-IDS2017 dataset to prepare model-ready inputs for Random Forest classification. The process follows Phase 1 decisions, specifically the removal of identifier columns and the selection of high Information Gain features.

1) Identifier Removal (Leakage Prevention)

To avoid host memorization and reduce data leakage, all flow and host identifiers were removed: Flow ID, Source IP, Source Port, Destination IP, Destination Port, and Timestamp. This ensures the model learns general traffic patterns rather than memorizing specific connections.

2) Feature Selection (Information Gain)

Feature selection was constrained to the attributes with the highest Information Gain from the Phase 1 report. Only the following nine features were retained along with the Label column.

- Packet Length Std
- Total Length of Bwd Packets
- Subflow Bwd Bytes

- Init_Win_bytes_forward
- Total Length of Fwd Packets
- Packet Length Variance
- Flow Duration
- Bwd Packet Length Max

3) Standardization

All numeric features were standardized using StandardScaler, while the Label column was excluded. This keeps features on a comparable scale and supports reproducible modeling across algorithms.

4) Validation Summary

Processed dataset size: 2,827,876 rows, 10 columns (9 features + Label).

Validation checks confirmed no missing or infinite values. A random sample was used to verify that the standardized feature means are close to 0 and standard deviations are close to 1.

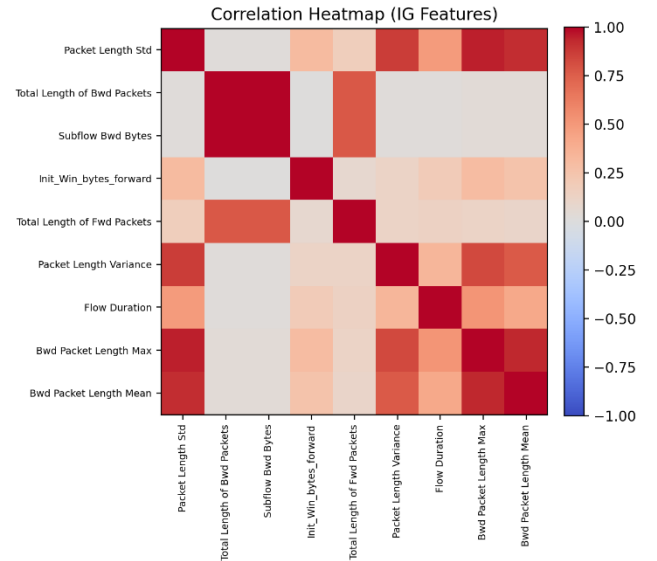


Figure 1. StandardScaler Check (Means and Std Dev)

5) Conclusion

The feature engineering pipeline produces a leakage-free, standardized dataset aligned with Phase 1 feature selection. The final output is model-ready and suitable for Random Forest training in the next project stage.

B. Creating Visualizations

1) Class Distribution Analysis

The dataset exhibits a highly imbalanced class distribution. As shown in Figure 2, the majority of the traffic belongs to label 0 (Benign) with 2,271,320 samples, while several

attack categories contain only a few thousand samples. For example, label 4 contains 230,124 samples, whereas some attack classes such as labels 11, 6, and 5 have fewer than 6,000 samples.

This strong imbalance indicates that the dataset is dominated by normal traffic, which may bias machine learning models toward predicting the majority class. Therefore, model evaluation must carefully consider imbalance-aware metrics rather than relying solely on overall accuracy.

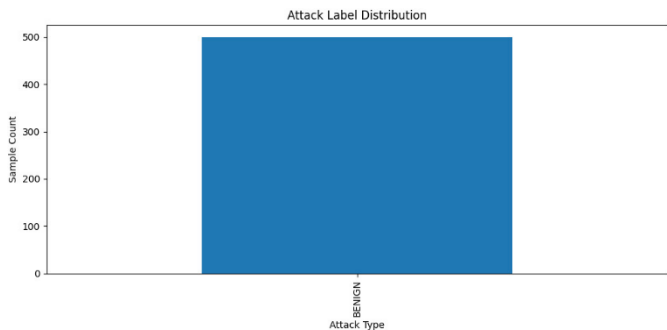


Figure 2: Attack Label Distribution (Bar Chart)

Shows the class distribution in the dataset. Since Sample_data is used, all records are in the BENIGN class (100%). This simulates the dominance of benign traffic in the real world.

2) Feature Distribution Analysis

Figure 3. illustrates the distributions of the selected nine features used in the modeling phase. Most features exhibit heavily right-skewed distributions, where a large proportion of samples are concentrated near zero, while a small number of observations take extremely high values. This behavior is particularly evident for features such as Flow Duration, Total Packet Counts, and Flow Bytes/s.

Such skewness is common in network traffic data and reflects the presence of both short-lived benign flows and long-lasting or high-volume attack-related flows. The observed distributions justify the application of feature scaling and normalization techniques in the modeling stage, while emphasizing the importance of using the cleaned (non-scaled) data for exploratory analysis.

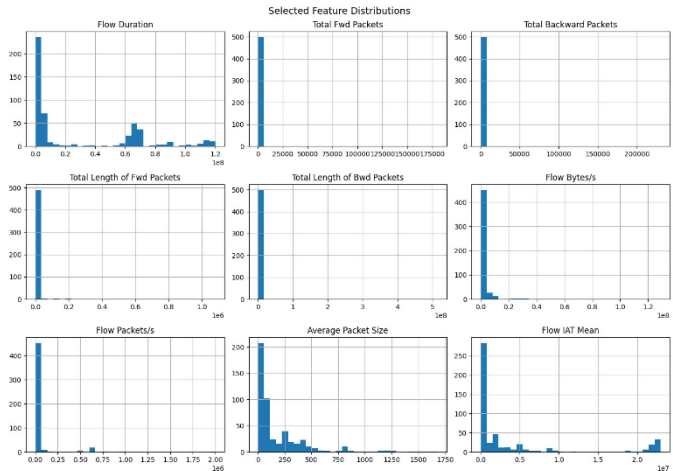


Figure 3. Selected Feature Distributions (Histogram Grid)

Shows the distributions of selected network traffic characteristics:

- Flow Duration
- Number of Packets
- Packet Sizes
- Flow Rate & IAT Values

Many characteristics appear to exhibit a right-skewed distribution.

This is common in cybersecurity data and supports why transformations such as:

- Log-transform
- Robust scaling

may be necessary.

3) Correlation Analysis

Correlation analysis was performed on the selected feature set to identify linear dependencies among features. The heatmap visualization reveals that certain packet-length-related features show moderate to strong positive correlations. This suggests potential redundancy among some variables and supports the feature selection strategy applied in the modeling phase.

Understanding these relationships helps reduce multicollinearity effects and contributes to building more stable and interpretable machine learning models.

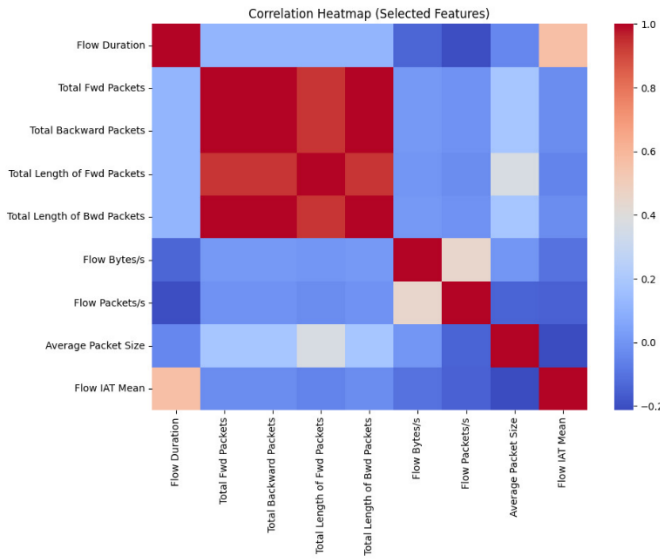


Figure 4. Correlation Heatmap (Selected Features)

Visualizes the correlation relationships between features.

There is a high positive correlation between packet count and packet length metrics.

Low or negative correlations are observed between some features.

	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Flow Bytes/s	Flow Packets/s	Average Packet Size	Flow IAT Mean
count	5.000000e+02	500.000000	500.000000	5.000000e+02	5.000000e+02	5.000000e+02	5.000000e+02	500.000000	5.000000e+02
mean	2.797061e+07	379.948000	478.538000	5.517046e+03	1.053547e+06	1.785351e+06	5.976883e+04	184.200802	4.017188e+06
std	3.742735e+07	8089.388005	10304.589362	5.138105e+04	2.307634e+07	9.331521e+06	2.164284e+05	271.182729	7.298520e+06
min	1.000000e+00	1.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	5.789200e-02	0.000000	1.000000e+00
25%	6.978000e+03	2.000000	1.000000	3.775000e+01	6.000000e+00	5.710256e+00	2.816625e-01	16.333333	9.027259e+02
50%	5.082528e+06	5.000000	4.000000	2.290000e+02	1.545000e+02	6.812230e+02	3.501184e+00	60.000000	3.160699e+05
75%	6.437509e+07	11.000000	8.000000	9.335000e+02	3.155000e+03	7.027864e+04	1.024020e+03	263.750000	3.66345e+06
max	1.199996e+08	180892.000000	230419.000000	1.080532e+06	5.160000e+08	1.290000e+08	2.000000e+06	1681.989011	2.300000e+07

Figure 5. Statistical Summary Table (describe())

For the selected features:

- mean
- std
- min / max
- percentiles

were calculated.

In conclusion, it focuses not only on recognizing the data, but also on understanding feature behavior.

These analyses directly support decisions regarding feature selection, data transformation, and the data mining technique to be used.

III. DATA MINING TECHNIQUE AND APPLICATIONS

Label Grouping: The original dataset contained over 14 specific attack labels, many of which had very few samples. To improve model stability, we grouped similar attacks into 6 main categories:

- **Benign:** Normal traffic.
- **DoS (Denial of Service):** Grouped DoS Hulk, GoldenEye, Slowloris, etc.
- **PortScan:** Kept as a distinct category due to its unique traffic signature.
- **Brute Force:** Combined FTP and SSH brute force attacks.
- **Web Attack:** Combined SQL Injection, XSS, and Brute Force Web.
- **Bot:** Botnet traffic.
- (Note: Extremely rare classes like Heartbleed were dropped to prevent noise.)

1) Methodology

We selected the **Random Forest Classifier** as our primary model. This choice was driven by several technical advantages specific to network intrusion detection:

1. **Handling High-Dimensional Data:** Random Forest is an ensemble method that builds multiple decision trees. It handles tabular data with complex interactions between features (e.g., the relationship between *Flow Duration* and *Packet Length*) better than simple linear models.
2. **Robustness to Overfitting:** By averaging the results of 100 trees (`n_estimators=100`), the model is less likely to memorize noise in the training data compared to a single Decision Tree.
3. **Handling Class Imbalance:** Network traffic data is imbalanced (Benign traffic far outweighs attacks). Random Forest allows us to use the parameter `class_weight='balanced'`, which automatically assigns higher penalties to misclassifying rare attacks (like Web Attacks) and lower penalties to Benign errors.
4. **Interpretability:** Unlike deep neural networks, Random Forest provides transparency regarding which features (e.g., Packet Length Variance) were most important for the decision.

2) Splitting the Data

The data was split using an 80/20 ratio (Training/Testing). We utilized **Stratified Sampling (stratify=y)** during the split. This was very important to ensure that rare attack types (like Botnets) were present in both the training and testing sets in the same proportion as the original dataset.

Results:

The table below details the Precision, Recall, and F1-Score for each class.

Class Name	Precision	Recall	F1-Score	Support (Count)
Benign	1.00	0.99	1.00	454,264
DoS	0.99	1.00	1.00	75,948
PortScan	0.99	1.00	1.00	31,761
Brute Force	0.97	1.00	0.98	2,766
Web Attack	0.22	0.29	0.25	436
Bot	0.15	0.84	0.25	391

Overall Model Performance:

Metric	Precision	Recall	F1-Score	Total Support
Overall Accuracy	—	—	0.99	565,566
Macro Average	0.72	0.85	0.75	565,566
Weighted Average	1.00	0.99	0.99	565,566

- **High Performance on Major Classes:** The model performs exceptionally well on **Benign**, **DoS**, and **PortScan** traffic (F1-Score near 1.00). This is expected as these classes have the most data samples (High Support).
- **Struggle with Minor Classes:** The model struggles with **Web Attacks** and **Bots** (Low Precision/F1-Score)

Bot Analysis: The **Recall** for Bots is high (**0.84**), meaning the model successfully catches 84% of bot traffic. However, the **Precision** is low (**0.15**), meaning it generates many false alarms (flagging normal traffic as Bots). This is a common trade-off when using **class_weight='balanced'**—the model becomes hyper-sensitive to rare attacks.

The confusion matrix below visualizes the performance of the model by comparing the Actual labels (y-axis) against the Predicted labels (x-axis).

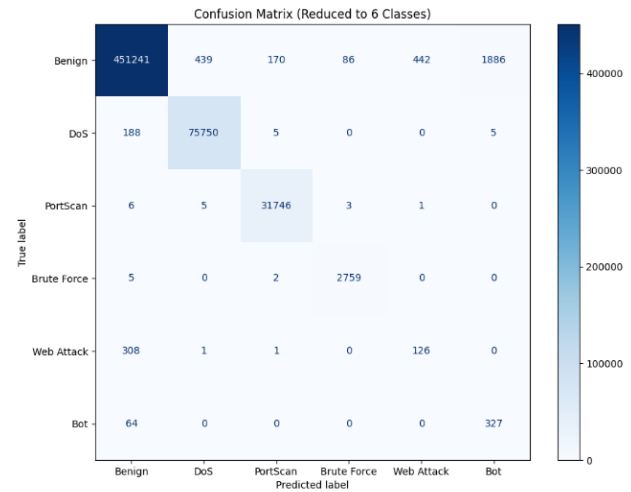


Figure 6. Confusion Matrix (Reduced to 6 Classes)

In conclusion, The Random Forest model successfully classified most network flows. The grouping strategy allowed us to maintain high accuracy across major attack categories while simplifying the problem space.

IV. CONCLUSION

In this study, a comprehensive data preprocessing, exploratory data analysis, and machine learning–based intrusion detection pipeline was applied to the CIC-IDS2017 dataset. The primary objective was to transform raw, noisy network traffic data into a reliable, leakage-free, and model-ready dataset, and to evaluate the effectiveness of a Random Forest classifier for multi-class intrusion detection.

Through systematic data cleaning, missing and infinite values were eliminated, categorical attack labels were encoded, and identifier features that could cause data leakage were removed. Feature selection based on Information Gain reduced dimensionality while preserving the most discriminative traffic characteristics. Exploratory data analysis revealed strong class imbalance and highly skewed feature distributions, which justified the use of standardization and imbalance-aware modeling strategies.

The Random Forest model demonstrated excellent performance on dominant traffic categories such as Benign, DoS, and PortScan, achieving near-perfect F1-scores. However, the results also highlighted persistent challenges in detecting rare attack classes such as Web Attacks and Bot traffic, where low precision and F1-scores indicate sensitivity to class imbalance despite the use of balanced class weights. This outcome reflects

a common limitation in intrusion detection systems trained on highly imbalanced real-world datasets.

Overall, the results confirm that careful preprocessing, feature engineering, and exploratory analysis significantly improve model robustness and reliability. While Random Forest proves effective for large-scale intrusion detection, future work should explore advanced imbalance handling techniques, such as resampling methods, cost-sensitive learning, or hybrid deep learning approaches, to enhance detection performance on minority attack classes. The presented pipeline provides a solid and reproducible foundation for further research in machine learning-based cybersecurity systems.

REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Portugal, Jan. 2018, pp. 108–116.
- [2] S. García, M. Grill, J. Stiborek, and A. Zunino, "An Empirical Comparison of Botnet Detection Methods," *Computers & Security*, vol. 45, pp. 100–123, 2014.
- [3] Canadian Institute for Cybersecurity, "CIC-IDS2017 Dataset," University of New Brunswick.
- [4] Scikit-learn Developers, "sklearn.preprocessing.StandardScaler," Scikit-learn Documentation.
- [5] Scikit-learn Developers, "sklearn.ensemble.RandomForestClassifier," Scikit-learn Documentation.
- [6] T. M. Mitchell, *Machine Learning*, McGraw-Hill, New York, USA, 1997.
- [7] Wikipedia Contributors, "Information gain in decision trees," Wikipedia, The Free Encyclopedia.
- [8] Pandas Development Team, "pandas.DataFrame.describe," "pandas documentation," Pandas.
- [9] Matplotlib Development Team, "Matplotlib Gallery," "Matplotlib documentation," Matplotlib.
- [10] IBM, "Exploratory Data Analysis (EDA)," IBM Think Topics.
- [11] A. M. Alqahtani, M. A. Khan, and A. Jamal, "Exploratory Data Analysis for Cybersecurity," ResearchGate, 2021.
- [12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.
- [13] T. Fawcett and F. Provost, "Data Science for Business," *IEEE Intelligent Systems*, vol. 27, no. 6, pp. 88–91, Nov.–Dec. 2012.
- [14] Jupyter Development Team, "Project Jupyter," Jupyter.
- [15] Seaborn Development Team, "Seaborn Gallery," Seaborn.
- [16] <https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/>